

"User level Thread"

User level threads are managed by User level library

User level threads are typically fast
Context Switching is faster

If One user level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

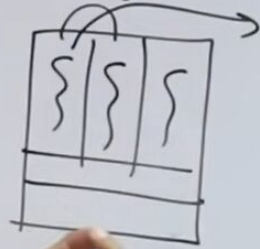
so kernel don't know about it
so kernel will block the whole thread

"User level Thread"

User level threads are managed by User level library

User level threads are typically fast
Context Switching is faster

If One user level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

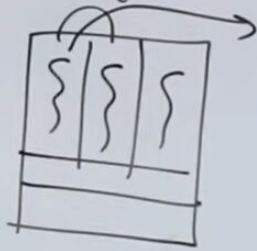
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

reason is that whole process will get block

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) If one user level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

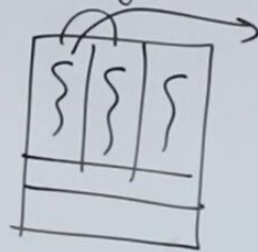
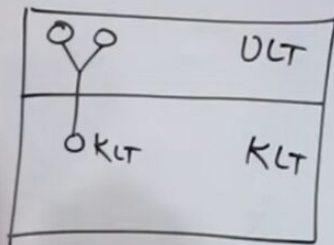
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

but if I'll talk about present time,
then we use hybrid environment

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) If one user level threads perform blocking operation then entire process get blocked



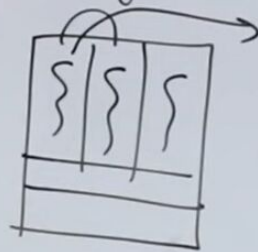
"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) If one user level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

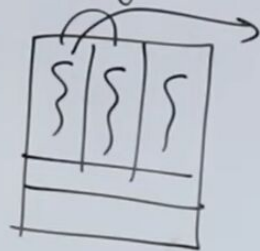
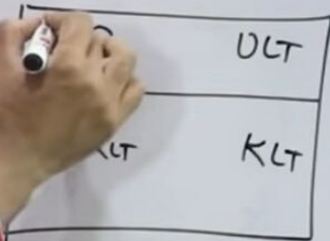
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

and we generally maps more than one user level threads with a kernel level thread

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) If one user level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

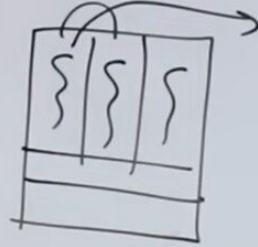
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

so here, this way we can resolve multiple threads with one kernel level

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) If one user level thread performs blocking operation then entire process gets blocked



"Kernel Level Thread"

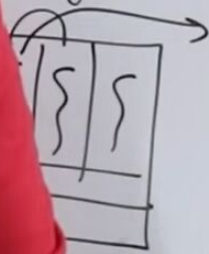
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread is blocked, No effect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

Which we call many to one, many to many...
we can use one to one also

"User level Thread"

- 1) User level threads are managed by User
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) User level threads perform blocking, process get blocked



"Kernel Level Thread"

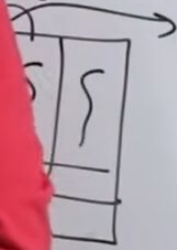
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

Like Linux uses one to one...
so that way we resolve problems in hybrid model

"User level Thread"

- 1) User level threads are managed by User
- 2) User level threads are typically fast
- 3) Context switching is faster
- 4) If one user level thread performs blocking, all threads in the process get blocked



"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{CT} > KLT > ULT$$

but talking about competitive exam,
so they generally ask difference between them only

"User level Thread"

Kernel threads are managed by User Library

Kernel threads are typically fast
Switching is faster

If one User level thread perform blocking
then entire process is blocked



"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

so this is the major difference

"User level Thread"

Kernel threads are managed by User library

Kernel threads are typically fast

Switching is faster

User level threads perform blocking
entire process get blocked



"Kernel Level Thread"

1) Kernel level threads are managed by OS System Calls

2) Kernel level threads are slower than User level

3) Context Switching is slower

4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{CT} > KLT > ULT$$

and let me tell you the last point
There arrived a question in GATE 2014

"User level Thread"

- 1) User level threads are managed by User space library
- 2) User level threads are typically fast
- 3) Context Switching is faster
- 4) User level threads perform blocking and the process gets blocked



"Kernel Level Thread"

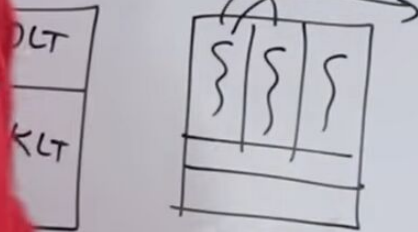
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

there was a point that kernel level thread
do not shares the code section

"User level Thread"

- 1) User level threads are managed by User
- 2) User level threads are typically fast
- 3) Context switching is faster
- 4) User level threads perform blocking when entire process get blocked



"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

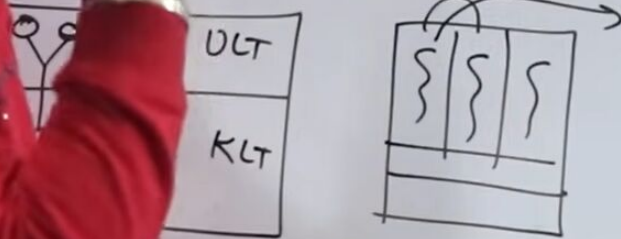
So let me clear it guys... whether it's kernel level thread or user level thread

"User level Thread"

User level threads are managed by User level library

User level threads are typically fast
Context Switching is faster

If User level threads perform blocking operation then entire process get blocked



"Kernel Level Thread"

- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{C_T} > KLT > ULT$$

It's a thread... and thread
always shares code and data

"User level Thread"

- 1) User level threads are managed by User level library
- 2) User level threads are typically fast
- 3) Context switching is faster
- 4) If one thread performs blocking operation, only that thread is blocked

Code
Data

"Kernel Level Thread"

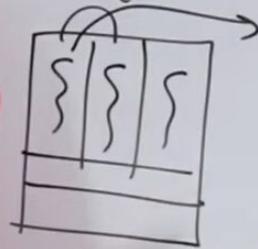
- 1) Kernel level threads are managed by OS System Calls
- 2) Kernel level threads are slower than User level
- 3) Context Switching is slower
- 4) If one Kernel level thread blocked, No affect on others.

$$\frac{\text{Process}}{CT} > KLT > ULT$$

but they have their own register setup and their own stack

"User level Thread"

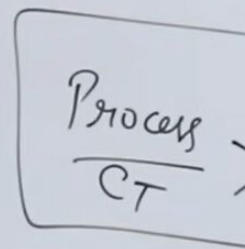
- 1) User level threads are managed by User level
- 2) User level threads are typically fast
- 3) Context switching is faster
- 4) User level threads perform blocking and entire process get blocked



Code
Data

"Kernel Level Thread"

- 1) Kernel level threads are managed by OS
- 2) Kernel level threads are typically fast
- 3) Context switching is faster
- 4) If one kernel thread performs blocking, only that thread is blocked, not the entire process.



Process
CT

so they share the rest of the portion
that's why we call them light weight process

Operating System (Complete Playlist)

3M+ VIEWS

Operating System Syllabus Discussion

92 videos

Important for all College/University & Competitive exams (GATE, NET)

L-2.1: Process Scheduling Algorithms (Preemption Vs Non-Preemption)

CPU Scheduling

Preemption vs Non-Preemption 10:31